

# Informática Gráfica

## Práctica 5: Interacción con ratón y selección de objetos



UNIVERSIDAD  
DE GRANADA

**Grado en Ingeniería Informática**

**Jesús Pereira Sánchez**

*jesuspereira@correo.ugr.es*

# Índice

|          |                                             |          |
|----------|---------------------------------------------|----------|
| <b>1</b> | <b>Introducción</b>                         | <b>2</b> |
| <b>2</b> | <b>Interacción con el Modelo Jerárquico</b> | <b>2</b> |
| 2.1      | Animaciones y estructura . . . . .          | 2        |
| 2.2      | Lógica . . . . .                            | 2        |
| 2.3      | Partes del Golem . . . . .                  | 3        |
| 2.3.1    | Head . . . . .                              | 4        |
| 2.3.2    | Arms . . . . .                              | 4        |
| 2.3.3    | Chest . . . . .                             | 5        |
| 2.3.4    | Legs . . . . .                              | 6        |

## 1. Introducción

En esta práctica 5 trabajaremos con la interacción del usuario mediante el ratón. Usando raycasting, podremos seleccionar objetos en nuestra escena 3D. Usaremos el modelo jerárquico de la práctica 3, al cual iremos seleccionando distintas partes para que se muevan.

## 2. Interacción con el Modelo Jerárquico

### 2.1. Animaciones y estructura

Antes de comenzar a explicar como se hace la interacción con el modelo, es necesario explicar cómo se ha realizado la estructura a nivel de scripts y programación con cada una de las partes del modelo jerárquico para poder llevar a cabo las animaciones.

En primer lugar, los estados. Según el estado en el que se encuentre el modelo, habrá una animación u otra. Tenemos:

1. **none**. Estado inicial. Modelo quieto.
2. **WALK**. Modelo caminando.
3. **ARM**. Movimiento de los brazos. Incluye a los 2 ejes de jerarquía.
4. **LEG**. Movimiento de las piernas en todo el eje de rotación.
5. **HEAD**. Movimiento de la cabeza y de los ojos.
6. **ALL**. Todos los movimientos a la vez.

Según se pulse una tecla o se seleccione una parte específica con el ratón, se cambiará de estado. Para evitar problemas de superposición entre estados, cada vez que se cambia de un estado a otro, el modelo reiniciará sus partes a las originales.

Cabe destacar que el código de los estados ha sido modificado respecto al de la práctica 3, con el fin de claridad en el mismo.

### 2.2. Lógica

La lógica que usaremos para la interacción será la ya presentada en el guión de prácticas para los modelos sencillos (torus, capsule). Para las partes interaccionables del modelo del golem motaremos la estructura jerárquica **MeshInstance3D > StaticBody3D > CollisionShape3D**.

Ahora, usaremos el código que ya tenemos de la cámara y el rayo que lanza. Actualizamos el código a lo siguiente:

```
1 func _input( event : InputEvent ) :  
2     # ... Previous code  
3  
4     if event is InputEventMouseButton and event.pressed and event.button_index ==  
5         MOUSE_BUTTON_LEFT:  
6         var mouse_pos = get_viewport().get_mouse_position()  
7         ray.global_position = project_ray_origin(mouse_pos)  
8         ray.target_position = project_local_ray_normal(mouse_pos) * ray_length
```

```

8 ray.force_raycast_update()
9 if ray.is_colliding():
10     var objeto = ray.get_collider()
11     print("Seleccionado: ", objeto.name)
12     match objeto.name:
13         "Floor":
14             crear_cubo_en(ray.get_collision_point())
15             print("Cubo creado en: ", ray.get_collision_point())
16         "Head":
17             moveHead.emit()
18         "Arms":
19             moveArms.emit()
20         "Legs":
21             moveLegs.emit()
22         "Walk":
23             walk.emit()
24         "Chest":
25             moveAll.emit()

```

Listing 1: Camera3DOrbital

Hemos extendido la funcionalidad previa. Cuando se detectaba que tocábamos el suelo "Floor" con el rayo, ejecutábamos una función. Bien, pues ahora cuando tocamos con el rayo las distintas partes del modelo, emitimos unas señales/eventos. De esta manera, nuestra cámara es el receptor.

Ahora, queda la parte del emisor y sus actuadores. Bien, pues cada parte (o nodo) es responsable de gestionar las señales que le llegan. En función de la señal que sea, el modelo activará un estado u otro. Y al activarlo, el modelo tendrá distintas animaciones.

El receptor captura de la siguiente manera las señales:

```

1 # ...
2 # Obteniendo al emisor para usar las senales que transmite
3 @onready var emisor = get_node("/root/EscenaPrincipal/Camera3DOrbital")
4
5 func _ready() -> void:
6     # Conectando las senales a los actuadores, que son las funciones que activan los estados
7     # de las distintas animaciones
8     emisor.moveHead.connect(_activateHead)
9     emisor.moveArms.connect(_activateArms)
10    emisor.moveLegs.connect(_activateLegs)
11    emisor.walk.connect(_activateWalk)
12    emisor.moveAll.connect(_activateAll)
13 # ...

```

Listing 2: Receptor

Las funciones actuadoras de cada señal cambian el estado del modelo, para las distintas animaciones.

En resumen, la detección y colisión se produce en el rayo que lanza la cámara. Cuando se detecta qué parte se trata el objeto colisionado, se lanza una señal, indicando al modelo que active ese estado y mueva una parte u otra.

## 2.3. Partes del Golem

Hemos dividido en varias partes al Golem, usando la estructura **MeshInstance3D > StaticBody3D > CollisionShape3D** para la colisión con el rayo. Es importante que nombremos al nodo StaticBody3D como toque, ya que al colisionar con el rayo en el script de la cámara usaremos su nombre para mandar distintas señales.

### 2.3.1. Head

El rayo cuando colisiona con la cabeza causa que esta se mueva, mandando la señal correspondiente. Resulta de la siguiente forma:

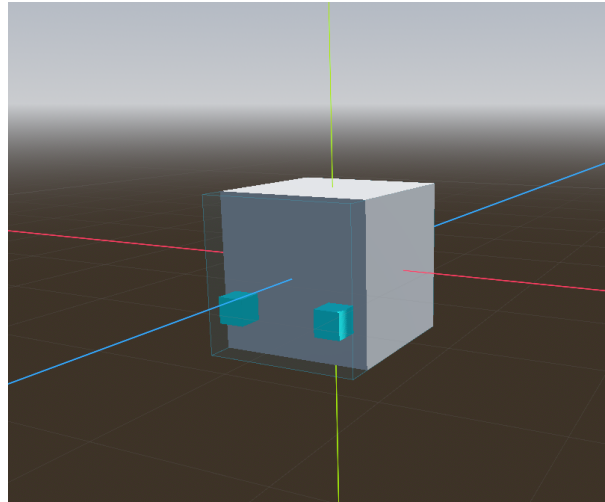


Figura 1: Head con collision3D

### 2.3.2. Arms

La señal que se manda es la del movimiento de los brazos en todos los ejes que puede. Uno de los brazos tendría la siguiente forma (se repite lo mismo que para el otro):

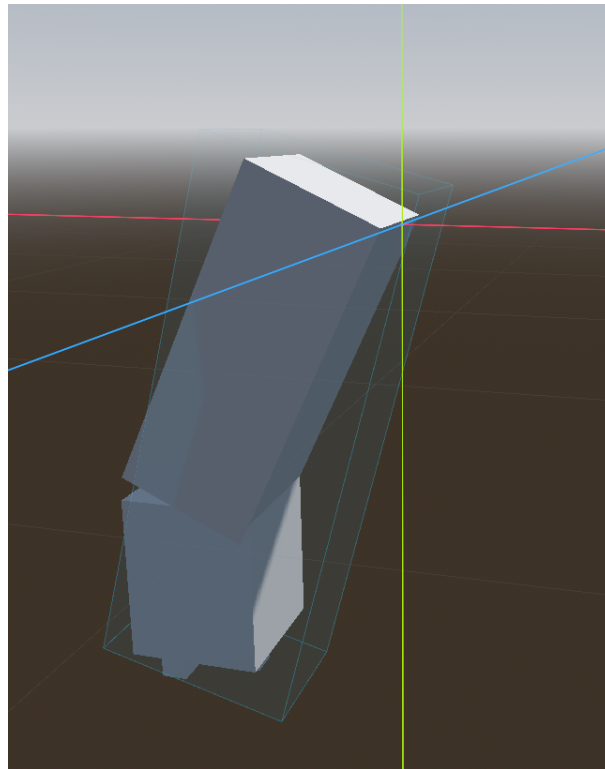


Figura 2: Arm con collision3D

### 2.3.3. Chest

La señal que se manda al clicar es la del movimiento de todas las partes según sus ejes. El collision3D del chest se ajusta a su forma rectangular:

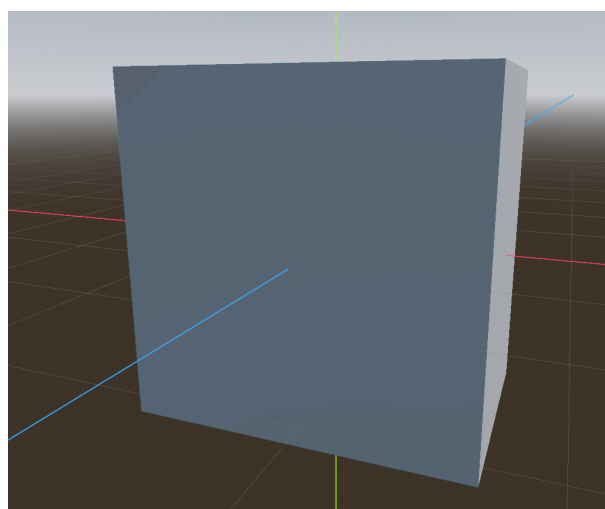


Figura 3: Chest con collision3D

### 2.3.4. Legs

En los legs hemos separado tanto los pies como los legs en sí. De esta manera, al colisionar con los legs estos se mueven con su señal, y al colisionar con los pies, se manda la señal de caminar (walk)

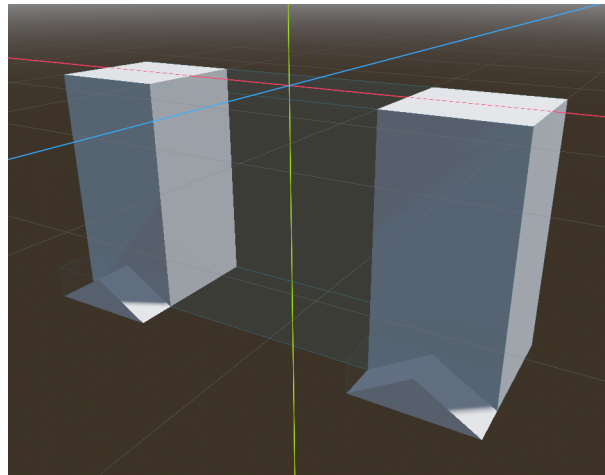


Figura 4: Legs con collision3D

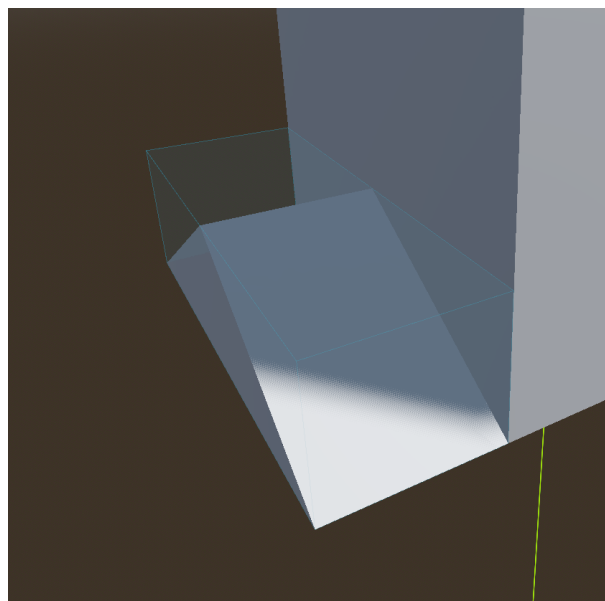


Figura 5: Feet con collision